

CxOrbil QA Engineer Intern Assignment

Date : 6 May 2026

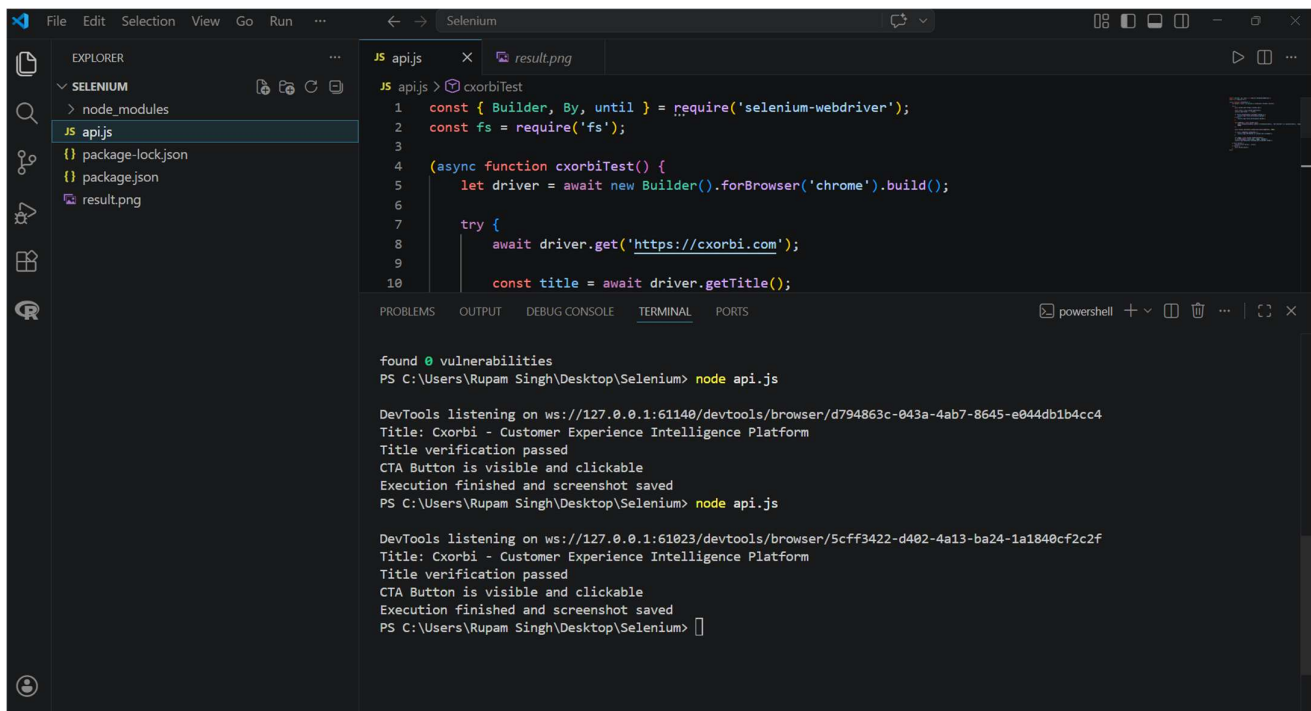
Created : Ravi Singh

Time : 2:06 PM

Task 1 : Write & Run an Automation Script

1. Title Verification : It retrieved the page title, which was "Cxorbi - Customer Experience Intelligence Platform". Since the code looked for the word "Cxorbi," the terminal confirmed that "Title verification passed".

2. CTA Button Check: It successfully found the primary Call to Action (CTA) button and verified it was both visible on the screen and ready to be clicked.



The screenshot displays the Visual Studio Code interface with a Selenium test script and its execution output. The Explorer panel on the left shows the project structure with files like `package-lock.json`, `package.json`, and `result.png`. The main editor shows the `api.js` file with the following code:

```
1 const { Builder, By, until } = require('selenium-webdriver');
2 const fs = require('fs');
3
4 (async function cxorbiTest() {
5     let driver = await new Builder().forBrowser('chrome').build();
6
7     try {
8         await driver.get('https://cxorbi.com');
9
10        const title = await driver.getTitle();
```

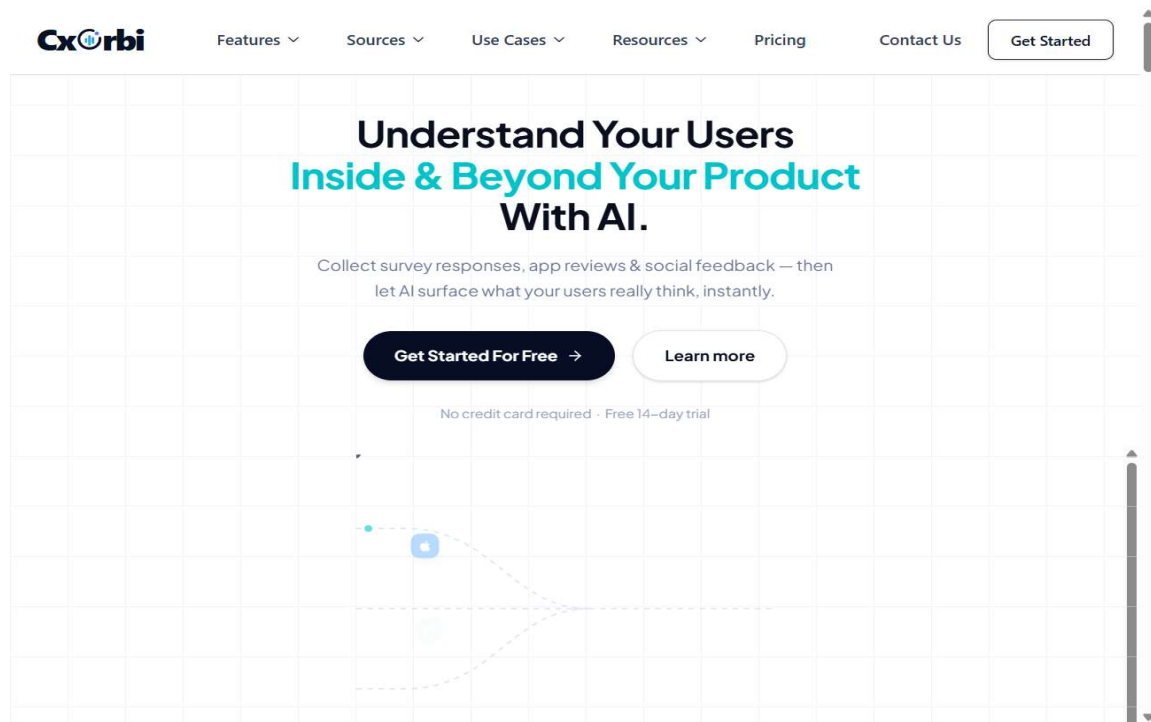
The terminal at the bottom shows the execution of `node api.js` and the resulting output:

```
found 0 vulnerabilities
PS C:\Users\Rupam Singh\Desktop\Selenium> node api.js

DevTools listening on ws://127.0.0.1:61140/devtools/browser/d794863c-043a-4ab7-8645-e044db1b4cc4
Title: Cxorbi - Customer Experience Intelligence Platform
Title verification passed
CTA Button is visible and clickable
Execution finished and screenshot saved
PS C:\Users\Rupam Singh\Desktop\Selenium> node api.js

DevTools listening on ws://127.0.0.1:61023/devtools/browser/5cff3422-d402-4a13-ba24-1a1840cf2c2f
Title: Cxorbi - Customer Experience Intelligence Platform
Title verification passed
CTA Button is visible and clickable
Execution finished and screenshot saved
PS C:\Users\Rupam Singh\Desktop\Selenium>
```

Result.png :



File link : <https://github.com/RaviSingh318/Intern-Assignment>

Task 2 — API Testing

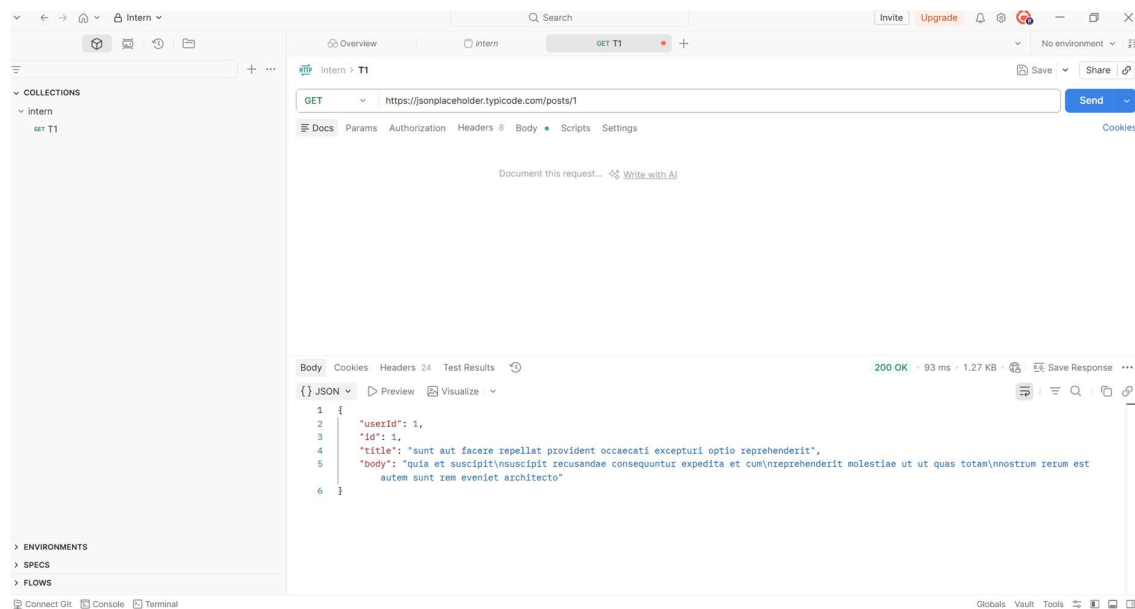
Test A:

GET <https://jsonplaceholder.typicode.com/posts/1>

→ Screenshot the response. What status code did you get?

Results

- Status Code : 200 OK
- The Response:



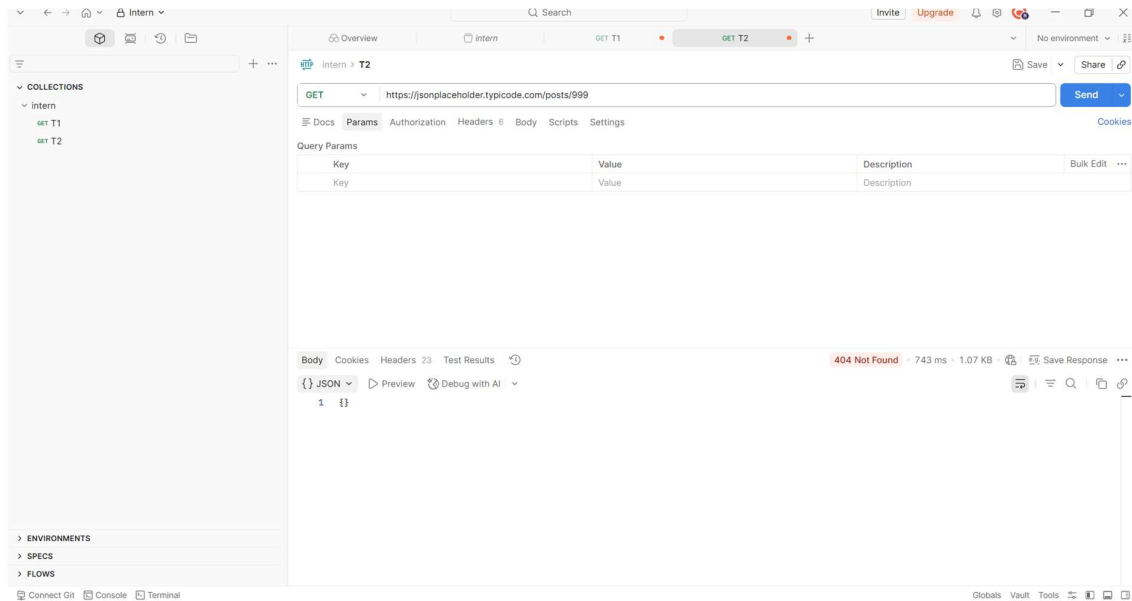
Test B:

GET <https://jsonplaceholder.typicode.com/posts/999>

→ Screenshot the response. What status code did you get? Is this the correct behavior? What should ideally happen?

Results

- Status Code : 404 Not Found
- The Response :



- Usually an empty object {} or a message indicating the resource doesn't exist
- The 400 Not Found status codes are reserved for "Client Errors." Because you (the client) requested a resource that simply isn't in the database, the server should correctly report that it cannot find it.

Test C:

POST <https://jsonplaceholder.typicode.com/posts>

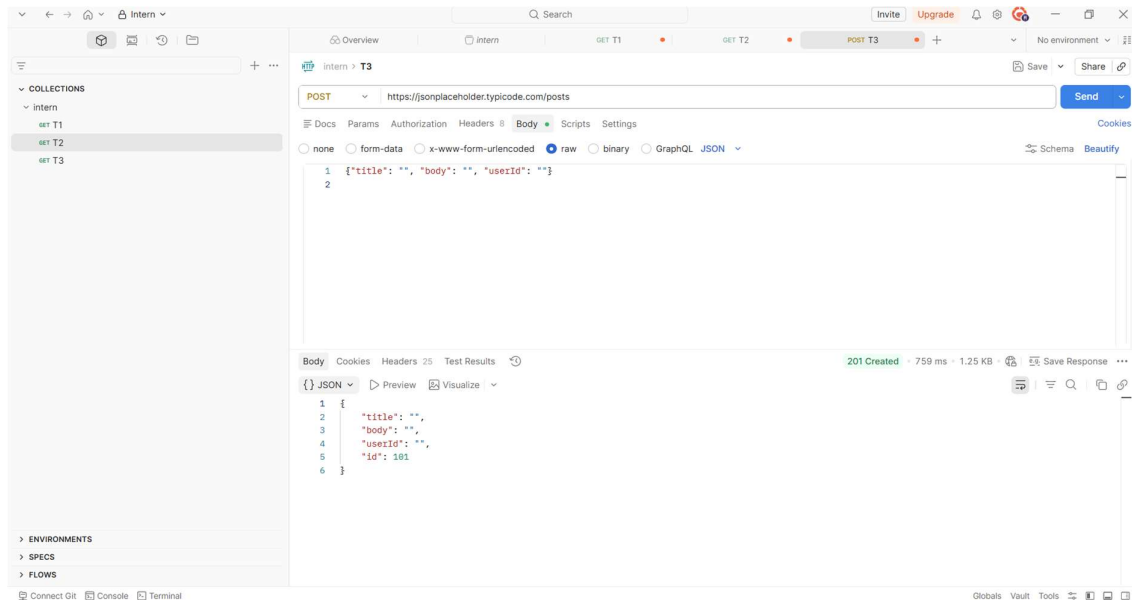
with this body:

```
{"title": "", "body": "", "userId": ""}
```

→ Screenshot the response. Did it succeed? Should an API accept completely empty fields? Explain why this is or isn't a problem.

Results

- Status Code : 201 Created
- The Response :



- Did it succeed : Technically, yes. The HTTP status code 201 indicates the resource was "created" on the server. this would be considered a logical failure.
- Should an API accept empty fields : No, API Layer that sits between the request and the database. If a request contains empty strings for required fields, the API should return a 400 Bad Request error.

Why is this a problem :

- **Data Integrity:** Your database will fill with useless, empty records that provide no value to users.
- **UI/UX Breakage :** If the API returns empty values, the website might show blank boxes, broken layouts, or throw errors when trying to process the missing data.
- **Security Risks:** Injected empty values can sometimes be used to bypass certain logic checks if the code assumes that if a key exists, it must have valid data.